Informatica® Cloud Data Integration

MongoDB V2 Connector

Informatica Cloud Data Integration MongoDB V2 Connector
October 2026

© Copyright Informatica LLC 2018, 2026

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica, the Informatica logo, Informatica Cloud, and PowerCenter are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices are included with the product.

See patents at https://www.informatica.com/legal/patents.html.

DISCLAIMER: Informatica LLC provides this documentation "as is" without warranty of any kind, either express or implied, including, but not limited to, the implied warranties of noninfringement, merchantability, or use for a particular purpose. Informatica LLC does not warrant that this software or documentation is error free. The information provided in this software or documentation may include technical inaccuracies or typographical errors. The information in this software and documentation is subject to change at any time without notice.

NOTICES

This Informatica product (the "Software") includes certain drivers (the "DataDirect Drivers") from DataDirect Technologies, an operating company of Progress Software Corporation ("DataDirect") which are subject to the following terms and conditions:

1. THE DATADIRECT DRIVERS ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT.
2. IN NO EVENT WILL DATADIRECT OR ITS THIRD PARTY SUPPLIERS BE LIABLE TO THE END-USER CUSTOMER FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL OR OTHER DAMAGES ARISING OUT OF THE USE OF THE ODBC DRIVERS, WHETHER OR NOT INFORMED OF THE POSSIBILITIES OF DAMAGES IN ADVANCE. THESE LIMITATIONS APPLY TO ALL CAUSES OF ACTION, INCLUDING, WITHOUT LIMITATION, BREACH OF CONTRACT, BREACH OF WARRANTY, NEGLIGENCE, STRICT LIABILITY, MISREPRESENTATION AND OTHER TORTS.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-10-15

Table of Contents

Preface 5

Informatica Resources 5

- Informatica Documentation 5
- IDMC web site 5
- IDMC Communities 5
- IDMC Marketplace 5
- Data Integration connector documentation 6
- Informatica Knowledge Base 6
- IDMC Trust Center 6
- IDMC Pulse Dashboard 6
- Informatica Global Customer Support 6

Chapter 1: Introduction to MongoDB V2 Connector 7

MongoDB V2 Connector assets 7

Chapter 2: Connections for MongoDB V2 8

Prepare for authentication 8

- Username and password 8
- X.509 standard 8
- LDAP 8

Connect to MongoDB 9

- Before you begin 9
- Connection details 9
- Authentication types 10
- Advanced settings 11

Additional connection properties 12

Configure SSL for the serverless runtime environment 14

Proxy server settings 15

Private links to access MongoDB 15

Chapter 3: Mappings for MongoDB V2 16

MongoDB V2 sources in mappings 16

- Joining multiple objects 18
- Adding and removing source fields 18
- Examples of advanced filter syntaxes 19
- Key range partitioning 19

MongoDB V2 targets in mappings 20

- Pass-through partitioning 22
- Rules and guidelines for MongoDB V2 target transformations 22

MongoDB V2 lookups in mappings 23

MongoDB V2 lookup properties. 23
Edit metadata in a MongoDB V2 source and target. 24
Reading and writing hierarchical data in mappings. 24
Stop on errors. 24
Capture rejected rows in an error file. 25
Capture changed data from CDC sources in a mapping. 26
Configuring a mapping task to read from a CDC source. 27
Configure additional properties. 28
Rules and guidelines for MongoDB V2 mappings. 29

Chapter 4: Data type reference. 30

MongoDB V2 and transformation data types. 30
Rules and guidelines for data type reference. 31

Index. 32

Preface

Use MongoDB V2 Connector to learn how to read from or write to MongoDB by using Data Integration. Learn to create a MongoDB V2 connection, develop and run mappings and mapping tasks in Cloud Data Integration.

Informatica Resources

Informatica provides you with a range of product resources through the Informatica Network and other online portals. Use the resources to get the most from your Informatica products and solutions and to learn from other Informatica users and subject matter experts.

Informatica Documentation

Use the Informatica Documentation Portal to explore an extensive library of documentation for current and recent product releases. To explore the Documentation Portal, visit https://docs.informatica.com.

If you have questions, comments, or ideas about the product documentation, contact the Informatica Documentation team at infa_documentation@informatica.com.

IDMC web site

You can access the Informatica® Intelligent Data Management Cloud™ (IDMC) web site at http://www.informatica.com/cloud.

IDMC Communities

Use the IDMC Community to discuss and resolve technical issues. You can also find technical tips, documentation updates, and answers to frequently asked questions.

Access the IDMC Community at:

https://network.informatica.com/community/informatica-network/products/cloud-integration

Developers can learn more and share tips at the Cloud Developer community:

https://network.informatica.com/community/informatica-network/products/cloud-integration/cloud-developers

IDMC Marketplace

Visit the Informatica Marketplace to try and buy Data Integration Connectors, templates, and mapplets:

https://marketplace.informatica.com/

Data Integration connector documentation

You can access documentation for Data Integration Connectors at the Documentation Portal. To explore the Documentation Portal, visit https://docs.informatica.com.

Informatica Knowledge Base

Use the Informatica Knowledge Base to find product resources such as how-to articles, best practices, video tutorials, and answers to frequently asked questions.

To search the Knowledge Base, visit https://search.informatica.com. If you have questions, comments, or ideas about the Knowledge Base, contact the Informatica Knowledge Base team at KB_Feedback@informatica.com.

IDMC Trust Center

The IDMC Trust Center provides information about Informatica security, compliance, and privacy practices.

You can access the trust center at https://trust.informatica.com/.

IDMC Pulse Dashboard

The IDMC Pulse Dashboard provides system status and upgrade information for all PODs and services across IDMC.

Follow the services you use and subscribe to alerts to receive notifications about upgrades, maintenance, production incidents, and upcoming releases. The IDMC Pulse Dashboard displays the production status of all the IDMC products. All maintenance updates are posted to this page, and during an outage, it will have the most current information. To ensure you are notified of updates and outages, you can subscribe to receive updates for a single service or all IDMC services. Subscribing to all services is the best way to be certain you never miss an update.

For more information, see the Service Status and Communications guide.

Informatica Global Customer Support

You can contact a Global Support Center through the Informatica Network or by telephone.

To find online support resources on the Informatica Network, click **Contact Support** in the IDMC Help menu to go to the **Cloud Support** page. The **Cloud Support** page includes system status information and community discussions. Log in to Informatica Network and click **Need Help** to find additional resources and to contact Informatica Global Customer Support through email.

The telephone numbers for Informatica Global Customer Support are available from the Informatica web site at https://www.informatica.com/services-and-training/support-services/contact-us.html.

CHAPTER 1

Introduction to MongoDB V2 Connector

You can use MongoDB V2 Connector to read data from or write data to collections in any on-premise or cloud deployment, such as MongoDB Atlas database.

You can use a MongoDB V2 connection as a source and as a target in mappings or mapping tasks.

You can also use a MongoDB V2 connection to read data from or write data to the following databases:

- Azure CosmosDB enabled with the MongoDB API in mappings and in mappings in advanced mode.
- Amazon DocumentDB in mappings in advanced mode.
- Other non-SSL MongoDB deployments in mappings in advanced mode.

The schema of the MongoDB V2 source is inferred by sampling rows from the MongoDB collection. MongoDB V2 Connector picks the most common data types from the sample for mixed records, sub-records, and arrays.

You can switch mappings to advanced mode to include transformations and functions that enable advanced functionality.

MongoDB V2 Connector assets

Create assets in Data Integration to integrate data using MongoDB V2 Connector.

When you use MongoDB V2 Connector, you can include the following Data Integration assets:

- Mapping
- Mapping task

For more information about configuring assets and transformations, see *Mappings*, *Transformations*, and *Tasks* in the Data Integration documentation.

CHAPTER 2

Connections for MongoDB V2

Create a MongoDB V2 connection to read data from or write data to collections in MongoDB. You can use a MongoDB V2 connection to access a MongoDB table, import MongoDB documents, preview data, and run mappings and mapping tasks.

Prepare for authentication

You can configure username and password, X.509 standard, and LDAP authentication types to access MongoDB.

Before you configure the connection properties, ensure that you have a MongoDB user account with admin privileges. You also need to copy the host name of the primary node in MongoDB and keep the authentication details handy based on the authentication type that you want to use.

Username and password

To connect to MongoDB using the username and password authentication, you need the MongoDB account user name, password, and database name.

Get the required details from your MongoDB account to which you want to connect.

For more information about the username and password authentication in MongoDB, see Authenticate a user with self-managed deployments.

X.509 standard

To connect to MongoDB using the X.509 standard authentication, you need the SSL keystore file, SSL keystore file password, and database name.

Get the required details from your MongoDB account to which you want to connect.

For more information about the X.509 standard authentication in MongoDB, see X.509 certificate.

LDAP

To connect to MongoDB using the LDAP authentication, you need the MongoDB account user name, password, and database name.

Before you use the LDAP authentication, complete the following tasks:

1. Log in to your MongoDB Atlas account.

2. Configure the LDAP server that you want to use.
3. Create the LDAP integration with the authentication domain, server URL, and credentials.

For more information about the LDAP authentication in MongoDB, see LDAP authorization on self-managed deployments.

Note: You can't use connections configured with the LDAP authentication in mappings in advanced mode.

Connect to MongoDB

Let's configure the MongoDB V2 connection properties to connect to MongoDB.

Before you begin

Before you get started, you'll need to get the relevant configuration details from your MongoDB account based on the authentication type that you want to configure.

Check out "Prepare for authentication" on page 8 to learn more about the authentication prerequisites.

Connection details

The following table describes the basic connection properties:

Property	Description
Connection Name	Name of the connection. Each connection name must be unique within the organization. Connection names can contain alphanumeric characters, spaces, and the following special characters: _ . + -, Maximum length is 255 characters.
Description	Description of the connection. Maximum length is 4000 characters.
Use Secret Vault	Stores sensitive credentials for this connection in the secrets manager that is configured for your organization. This property appears only if secrets manager is set up for your organization. When you enable the secret vault in the connection, you can select the credentials that the Secure Agent retrieves from the secrets manager. If you don't enable this option, the credentials are stored in the repository or on a local Secure Agent, depending on how your organization is configured. **Note:** If you're using this connection to apply data access policies through pushdown or proxy services, you cannot use the Secret Vault configuration option. For information about how to configure and use a secrets manager, see "Secrets manager configuration" in the Administrator help.
Runtime Environment	The name of the runtime environment where you want to run tasks. Select a Secure Agent, serverless, or elastic runtime environment. For more information about runtime environments, see *Runtime Environments* in the Administrator help.

Property	Description
Host	Node name or IP address of the primary shard in the MongoDB cluster. If you use a serverless runtime environment with AWS PrivateLink, enter the direct node address. For more information, see "Private links to access MongoDB" on page 15.
Service Record Lookup Enabled	Enables the connector to query the DNS and discover the available MongoDB servers in the MongoDB cluster. You can enable this option when the host name is associated with a DNS SRV record. If you enable this option, the Secure Agent uses the required connection details from the DNS SRV record instead of the port number. Retain the default port number, 27017, to save the connection. If you don't enable this option, manually specify the host name and port number of the MongoDB server.
Port	MongoDB server port number. Default is 27017.

Authentication types

You can configure username and password, X.509 standard, and LDAP authentication types to access MongoDB.

Select the required authentication type and then configure the authentication-specific parameters.

Username and password authentication

Username and password authentication is the default type which requires at a minimum your MongoDB account user name, password, and database name to connect to MongoDB.

The following table describes the basic connection properties for username and password authentication:

Property	Description
User Name	User name to access the MongoDB server.
Password	Password for the user name to access the MongoDB server.
Database Name	Name of the MongoDB database that you want to connect to.

X.509 standard authentication

X.509 standard authentication requires your SSL keystore file path and SSL keystore file password to use X.509 certificate to connect to the MongoDB server.

The following table describes the basic connection properties for X.509 standard authentication:

Property	Description
SSL KeyStore File Path	The absolute path of the keystore file in the Secure Agent machine that contains the keys and certificates required to establish a secure communication. Ensure that you download the certificates and place them in the Secure Agent machine before you specify this parameter. For the serverless runtime environment, specify the following certificate path in the serverless agent directory: `/home/cldagnt/SystemAgent/serverless/configurations/ssl_store/<keystore_filename>` For more information, see "Configure SSL for the serverless runtime environment" on page 14.
SSL KeyStore Password	The password for the keystore file required for secure communication.
Database Name	Name of the MongoDB database that you want to connect to.

LDAP authentication

The minimum requirement for an LDAP authentication is the MongoDB user name, password, and database name to connect to MongoDB.

The following table describes the basic connection properties for LDAP authentication:

Property	Description
User Name	User name to access the MongoDB server.
Password	Password for the user name to access the MongoDB server.
Database Name	Name of the MongoDB database that you want to connect to.

Advanced settings

The following table describes the advanced connection properties:

Property	Description
Additional Properties	Optional properties that you can configure to read data from or write data to MongoDB. For information about the additional properties that you can configure, see "Additional connection properties" on page 12. To specify more than one property, separate the key-value pairs with an ampersand. You can specify the properties in the following format: *propertyName1=<value1>&propertyName2=<value2>*

Additional connection properties

You can configure certain additional options in the **Additional Properties** field in a MongoDB V2 connection that apply for a mapping or a mapping in advanced mode.

Sampling properties

You can configure the following sampling properties in a mapping or a mapping in advanced mode:

samplesize

The number of documents to scan to infer the schema from the MongoDB source.

For example, `samplesize=100`.

Default is 100.

samplemethod

The method to sample documents to infer the schema from the MongoDB source.

You can specify one of the following methods:

- firstpage. Scans first n documents from MongoDB where n indicates the sample size. MongoDB determines the ordering of rows for scanning.
- random. Scans n number of random documents from MongoDB.
- all. Scans the entire collection to infer the schema.
- none. Scans the entire collection to write multi-level hierarchical data without flattening from a MongoDB source to a MongoDB target in a mapping. This feature does not apply to mappings in advanced mode.
  You can write to an empty MongoDB collection only when you specify `none` as the `samplemethod`.

Non-SSL MongoDB deployments

You can configure the following additional properties to connect to non-SSL MongoDB deployments in a mapping or a mapping in advanced mode:

ssl

Determines if the connection uses SSL or non-SSL.

Set this parameter to false in the connection properties to connect to MongoDB deployments that do not use SSL.

Default is true.

authsource

Allows you to provide the database name against which you can authenticate user credentials.

For example, `authsource=testadmin`.

Default is admin.

Amazon DocumentDB

You can configure the following additional properties to connect to Amazon DocumentDB in a mapping in advanced mode:

ssltruststorefilepath

The absolute path of the truststore file in the Secure Agent machine that contains the keys and certificates required to establish a secure communication.

For example, `ssltruststorefilepath= <path_of_truststore_file>`

For the serverless runtime environment, specify the following certificate path to the serverless agent directory:

`/home/cldagnt/SystemAgent/serverless/configurations/ssl_store/<truststore_filename>`

For more information, see "Configure SSL for the serverless runtime environment" on page 14.

ssltruststorepassword

The password for the truststore file required for secure communication.

For example, `ssltruststorepassword=<password>`.

Bypass schema validation

You can configure the following additional properties to bypass validation rules set for a MongoDB collection in a mapping:

bypassdocumentvalidation

Determines if the connection bypasses validation rules set for a MongoDB collection when you insert, update, or upsert data to MongoDB.

For example, to bypass validation rules, enter `bypassdocumentvalidation=true`.

If you don't set this parameter, the MongoDB V2 connection uses validation rules defined for the MongoDB collection when you write data to MongoDB.

Verify that you have either the dbAdmin or restore privilege on MongoDB before you bypass validation rules set for a MongoDB collection.

For more information, see Bypass schema validation in the MongoDB documentation.

Azure CosmosDB

You can configure the following additional properties to write to Azure CosmosDB enabled with the MongoDB API in a mapping:

retryWrites

Determines if a mapping can automatically retry attempts to write to Azure CosmosDB.

For example, to write to Azure CosmosDB, enter `retryWrites=false`.

If you set this parameter, the target connection disables the automatic retry mechanism when you write to Azure CosmosDB and you can't configure validation rules for an Azure CosmosDB collection.

If you don't set this parameter, the target connection enables the automatic retry mechanism by default.

Default is true.

You don't need to set this property to false when you read from Azure CosmosDB.

For more information, see Retryable Writes in the MongoDB documentation.

Handle columns with null values

You can configure the following additional properties to handle columns with null values when writing data to a MongoDB V2 target in a mapping or a mapping in advanced mode:

OmitNullColumns

Determines whether a mapping omits columns with null values when writing to a MongoDB V2 target.

For example, to omit columns with null values, enter `OmitNullColumns=true`.

If you don't set this parameter, the mapping writes columns with null values to the target.

Default is false.

If you set this parameter, you can't configure the `samplemethod` additional property value to `none`.

SkipRowsWithNullUpdateColumns

Determines how a mapping handles the write operation on the target when update columns contain null or missing values.

If you set this parameter, the mapping skips updating columns that contain null or missing values, which prevents bulk updates to the target with undefined data.

For example, enter `SkipRowsWithNullUpdateColumns=true`.

If you don't set this parameter, the mapping updates columns even if they contain null or missing values.

Default is false.

If you set this parameter, you can't configure the `samplemethod` additional property value to `none`.

Configure SSL for the serverless runtime environment

You can use the serverless runtime environment with MongoDB V2 Connector to connect to an SSL-enabled MongoDB database in mappings in advanced mode.

Before you configure a secure MongoDB V2 connection using the serverless runtime environment, you need to perform certain prerequisites:

1. Ensure that the truststore and keystore certificate files are in JKS format.
2. Add the truststore and keystore certificates in the Amazon S3 bucket or Azure container in the following location in your AWS or Azure account: `<Supplementary file location>/serverless_agent_config/SSL`
3. Copy the following code snippet to a text editor:

```
version: 1
agent:
 agentAutoApply:
    general:
      sslStore:
        - fileCopy:
            sourcePath: SSL/<cert_filename>
        - fileCopy:
            sourcePath: SSL/<cert_filename>
```

   where the source path is the directory path of the certificate files on AWS or Azure.

   Note: You can add multiple source paths of the certificate files by adding multiple *fileCopy* tags.
4. Ensure that the syntax and indentations are valid, and then save the file as `serverlessUserAgentConfig.yml` in the following AWS or Azure location: `<Supplementary file location>/serverless_agent_config`
   When the .yml file runs, the SSL certificates are copied from the AWS or Azure location to the serverless agent directory.
5. Deploy the serverless agent.
6. Specify the following certificate path in the serverless agent directory for the truststore and keystore file path fields: `/home/cldagnt/SystemAgent/serverless/configurations/ssl_store/<cert_filename>`

Proxy server settings

If your organization uses an outgoing proxy server to connect to the Internet, the Secure Agent connects to Informatica® Intelligent Data Management Cloud™ (IDMC) through the proxy server.

You can configure the Secure Agent to use the SOCKS proxy server on Windows and Linux. You can use the unauthenticated or authenticated SOCKS proxy server. The proxy settings apply to connections used in mappings and in mappings in advanced mode.

To configure proxy settings for the Secure Agent, contact Informatica Global Customer Support.

Private links to access MongoDB

You can configure AWS PrivateLink to enable private communication between the serverless runtime environment hosted on AWS platform and a MongoDB endpoint.

To configure AWS PrivateLink using a serverless runtime environment, specify the direct node address in the **Advanced Custom Properties** field in the serverless runtime environment properties. Then, specify the same direct node address as the host name in the **Host** field in the MongoDB V2 connection, instead of entering the MongoDB cluster name, which is the primary node.

For more information about configuring a serverless runtime environment using AWS PrivateLink, see "Serverless runtime environments" in *Runtime Environments* in the Administrator help.

CHAPTER 3

Mappings for MongoDB V2

You can configure a mapping to read from and write to MongoDB tables.

Use the Mapping Designer in Data Integration to add the Source or Target transformations in the mapping canvas and configure the MongoDB V2 source and target properties.

In advanced mode, the Mapping Designer updates the mapping canvas to include transformations and functions that enable advanced functionality. When you run a mapping in advanced mode, Data Integration analyzes the data logic defined in the mapping to automatically assign data logic to run on an advanced cluster

You can use Monitor to monitor the job.

MongoDB V2 sources in mappings

To read data from MongoDB, configure a MongoDB V2 object as the Source transformation in a mapping.

Specify the name and description of the MongoDB V2 source. Configure the source, query options, and advanced properties for the source object. When you configure a Source transformation, you can add or remove source fields of primitive data types.

You can parameterize the MongoDB V2 source connection and source object with values specified in a parameter file when you configure a mapping or a mapping in advanced mode.

The following table describes the source properties that you can configure for a MongoDB V2 source:

Property	Description
Connection	Name of the MongoDB V2 source connection. You can select an existing connection or create a new connection. You can also click **New Parameter** to define a parameter for the source connection.
Object Type	Type of the source object. Select one of the following options: - Single Object - Multiple Objects[1] - Parameter Related objects appear based on simple primary key-foreign key relationships. You can use the Advanced Relationship option to define the relationship for objects that you want to join.

Property	Description
Object	Name of the MongoDB V2 source object based on the object type selected. Select the source object for a single source.
[1] Applies only to mappings in advanced mode.	

The following table describes the query options that you can configure in a Source transformation:

Property	Description
Filter	Filter value in a read operation. Click **Configure** to add conditions to filter records and reduce the number of rows that the Secure Agent reads from the source. You can specify the following filter conditions: - **Not parameterized**. Use a basic filter to specify the object, field, operator, and value to select specific records. - **Completely parameterized**. Use a parameter to represent the field mapping. - **Advanced**. Use an advanced filter to define a more complex filter condition on an object. For more information about syntaxes of the complex filter conditions, see "Examples of advanced filter syntaxes" on page 19. You can configure filter conditions for columns of the following data types: - Date - Double - Float - Integer - String
Select distinct rows only	Not applicable.

The following table describes the advanced properties that you can configure in a Source transformation:

Property	Description
Pre Query[1]	The SQL statement that you want to run before reading data from the source. You can use insert, update, and delete operations in the SQL query. For example, if you want to insert records into the database before reading records from a table, specify the following preSQL statement: `{ insert: 'yourCollection', documents: [ { name: 'Alice', age: 25 } ] }`
Post Query[1]	The SQL statement that you want to run after reading data from the source. You can use insert, update, and delete operations in the SQL query. For example, if you want to update records into a table after reading records from a table, specify the following postSQL statement: `{ update: 'yourCollection', updates: [ { q: { name: 'Alice' }, u: { $set: { age: 26 } } } ] }`
Additional Properties	Additional options that you want to specify to read data from MongoDB. Separate multiple options with an ampersand. For information about the additional properties that you can configure, see the "Configure additional properties" on page 28 chapter.

Property	Description
Tracing Level	Amount of detail that appears in the log for this transformation. You can use the following tracing levels: - Terse - Normal - Verbose Initialization - Verbose Default is normal.
[1] Doesn't apply to mappings in advanced mode.	

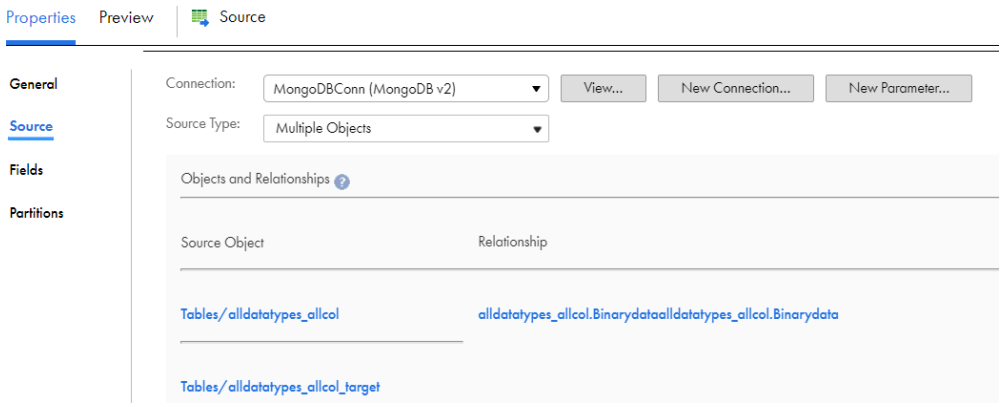
Joining multiple objects

When you create a Source transformation to run mappings in advanced mode, you can select multiple MongoDB V2 objects as the object type and then configure an advanced relationship to combine the tables.

Perform the following steps to join multiple objects in a Source transformation:

1. In the Source transformation properties, select the **Object Type** as **Multiple Objects**.
2. From the **Actions** menu, select **Add Source Object**, and then select the source object that you want to add from the displayed list.
3. From the **Related Objects Actions** menu, select the Advanced Relationship option to define the relationship between the tables.

   The following image shows an advanced relationship condition defined between MongoDB tables:

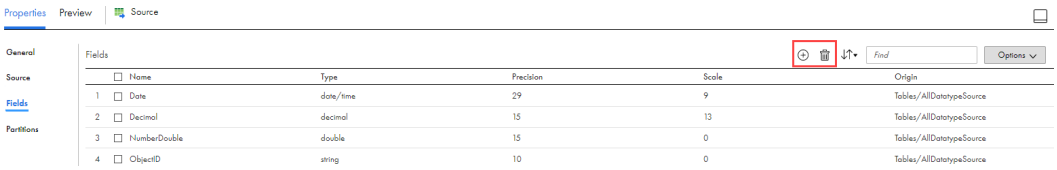


Note: You can join only two source tables. You must define only the join condition, not the complete join query.

Adding and removing source fields

You can configure the source fields that you want to use in the data flow.

You can add fields to a mapping source and retrieve a field from the source object that is not displayed in the list. To add a field, click ⊕, and then enter the field name, type, precision, and scale. You can also remove fields that you do not want to use in the mapping. To remove a field, select the field and then click 🗑.



Examples of advanced filter syntaxes

Consider the following examples of filter syntaxes when you use an advanced filter:

- `Equals - { "field": "value" }`
- `Not equal - { "field": { "$ne": "value" } }`
- `Greater than - { "field": { "$gt": "value" } }`
- `Greater than or equal - { "field": { "$gte": "value" } }`
- `Less than - { "field": { "$lt": "value" } }`
- `Less than or equal - { "field": { "$lte": "value" } }`
- `Combines filters with logical AND - { "$and": [{ "field1": { "$gt": "value1" } }, { "field2": { "$lt": "value2" } }] }`
- `Combines filters with logical OR - { "$or": [{ "field1": "value1" }, { "field2": "value2" }] }Inverts a filter - { "field": { "$not": { "$eq": "value" } } }`
- `Matches documents with the specified BSON type - { "field": { "$type": "string" } } / { "field": { "$type": "int" } }`
- `Matches if field's value is in the provided array - { "field": { "$in": ["value1", "value2", "value3"] } }`
- `Matches if field's value is not in the provided array - { "field": { "$nin": ["value1", "value2", "value3"] } }`
- `Regular expression matching - { "field": { "$regex": "pattern" } } / { "field": { "$regex": "pattern", "$options": "i" } }`

Key range partitioning

You can configure key range partitioning when you use mapping tasks to read data from MongoDB V2 sources. With key range partitioning, the Secure Agent distributes rows of source data based on the fields that you define as partition keys. The Secure Agent compares the field value to the range values for each partition and sends rows to the appropriate partitions.

Use key range partitioning for columns that have an even distribution of data values. Otherwise, the partitions might have unequal size. For example, a column might have 10 rows between key values 1 and 1000 and the column might have 999 rows between key values 1001 and 2000. If the mapping includes multiple sources, use the same number of key ranges for each source.

When you define key range partitioning for a column, the Secure Agent reads the rows that are within the specified partition range. For example, if you configure two partitions for a column with the ranges as 10 through 20 and 30 through 40, the Secure Agent does not read the rows 20 through 30 because these rows are not within the specified partition range.

You can configure a partition key for fields of the following data types:

- Date
- Decimal128

- Double
- Int32
- Int64
- Integer
- String

You can't use key range partitioning for mapping tasks that are based on mappings in advanced mode.

For more information about partitions, see *Transformations* in the Data Integration documentation.

MongoDB V2 targets in mappings

To write data to a MongoDB V2 target, configure a MongoDB V2 object as a Target transformation in a mapping.

Specify the name and description of a MongoDB V2 target.

You can parameterize the MongoDB V2 target connection and target object with values specified in a parameter file when you configure a mapping or a mapping in advanced mode.

The following table describes the target properties that you can configure for a MongoDB V2 target:

Property	Description
Connection	Name of the MongoDB V2 connection that is associated with a dataset. You can select an existing connection or create a new connection. You can also click New Parameter to define a parameter for the target connection.
Object Type	Type of the target object. Select one of the following options: - Single Object - Parameter
Object	Name of the MongoDB V2 target object based on the object type selected.
Existing	You can select an existing MongoDB V2 target table to write data. When you use an existing target, you can select one of the following schema options: - Infer schema from the selected target object[2]. Select only when you read primitive data types. - Generate the schema for the target object from the incoming fields to the Target transformation[1]. Select when you read primitive or hierarchical data types.
Create New at Runtime	Creates a MongoDB V2 target table at runtime based on the target table you specify. To create a target table at runtime, enter the name of the target table that you want to create in the **Object Name** field. The agent creates the target table based on the object name and the path you specify. You can edit the metadata of the source fields before creating the target. When you create a new target at run time and the target already exists, the mapping appends the data to the existing target.

Property	Description
Operation	Type of the target operation. Select one of the following operations: - Insert - Update - Delete - Upsert[2] - Data Driven[2] Default is Insert.
Update Columns	The primary key column to update, upsert, or delete data in a MongoDB target. This property appears only when you select **Update**, **Delete**, **Upsert**, or **Data Driven** as the operation type.
Data Driven Condition[2]	Enables you to define expressions that flag rows for an insert, update, upsert, or delete operation. This property appears only when you select **Data Driven** as the operation type. You need to specify the data driven condition for non-CDC sources. For CDC sources, you need to leave the field empty as the rows in the CDC source tables are already marked with the operation types.
[1] Applies only to mappings in advanced mode. [2] Doesn't apply to mappings in advanced mode.	

The following table describes the advanced properties that you can configure in a Target transformation:

Property	Description
Update Mode[1]	Loads data to the target based on the mode you specify. Select from one of the following options: - Update As Update: Rows matching the selected update columns are updated in the target. - Update Else Insert: Rows matching the selected update columns are updated in the target. Rows that don't match are appended to the target.
Truncate Target[1]	Select to truncate the target collection by removing all documents from the MongoDB collection before writing data to the target. If you don't select this option, the mapping writes data to the target without truncating the MongoDB target collection. Default is disabled.
Pre Query[1]	The SQL statement that you want to run before writing data to the target. You can use insert, update, and delete operations in the SQL query. For example, if you want to insert records into the database before writing records to a table, specify the following preSQL statement: `{ insert: 'yourCollection', documents: [ { name: 'Alice', age: 25 } ] }`
Post Query[1]	The SQL statement that you want to run after writing data to the target. You can use insert, update, and delete operations in the SQL query. For example, if you want to update records in a table after writing records to a table, specify the following postSQL statement: `{ update: 'yourCollection', updates: [ { q: { name: 'Alice' }, u: { $set: { age: 26 } } } ] }`

Property	Description
Additional Properties	Additional options that you want to specify to write data to MongoDB. Separate multiple options with an ampersand. For information about the additional properties that you can configure, see the "Configure additional properties" on page 28 chapter.
Forward Rejected Rows	Determines whether the transformation passes rejected rows to the next transformation or drops rejected rows. By default, the agent forwards rejected rows to the next transformation.
[1] Doesn't apply to mappings in advanced mode.	

Pass-through partitioning

You can configure pass-through partitioning along with other target properties when you write data to MongoDB V2 targets.

The pass-through partitioning method controls how the agent distributes data among partitions at partition points. You can define the partition type as pass-through partitioning. With partitioning, the agent distributes rows of target data based on the number of threads that you define as partition.

You can't use pass-through partitioning for mapping tasks that are based on mappings in advanced mode.

For more information about partitions, see *Transformations* in the Data Integration documentation.

Rules and guidelines for MongoDB V2 target transformations

Consider the following guidelines for MongoDB V2 target transformations:

- When you perform an insert operation, the Secure Agent inserts and updates the entries in the target table based on the source key value and target key value match. The matching criteria for insert operation is *_id*.
- When you perform an update operation, you can select update columns only with primitive data types.
- When you perform an update or delete operation, the *_id* column in the target table is considered as a key column along with the other update columns selected.
- When you create a new target at run time and the target already exists, the mapping updates the data to the existing target when you perform an update operation and deletes the data from the existing target when you perform a delete operation.
- To perform an upsert operation on the target, set the update mode to **Update Else Insert** in the Target transformation.
- To perform a Data Driven operation on the target, specify a column in the **Update Columns** field even if you defined a primary key in the target table.
- When you override the parameterized target object at run time in a mapping, you can't perform the Data Driven operation on the target object.

MongoDB V2 lookups in mappings

Add a Lookup transformation to retrieve data based on a specified lookup condition. When you add a Lookup transformation to a mapping or a mapping in advanced mode, you define the lookup connection, lookup objects, and lookup properties related to MongoDB.

You can look up MongoDB data values based on a condition that you configure. In the Lookup transformation, select the lookup connection and object. Then, define the lookup condition and the outcome for multiple matches.

The mapping queries the lookup source based on the lookup fields and the defined lookup condition. The lookup operation returns the result to the Lookup transformation, which then passes the results downstream.

You can parameterize the MongoDB V2 lookup connection and lookup object with values specified in a parameter file when you configure a mapping or a mapping in advanced mode.

You can configure the cached connected and unconnected lookups for mappings or mappings in advanced mode.

For more information about Lookup transformation, see *Transformations* in the Data Integration documentation.

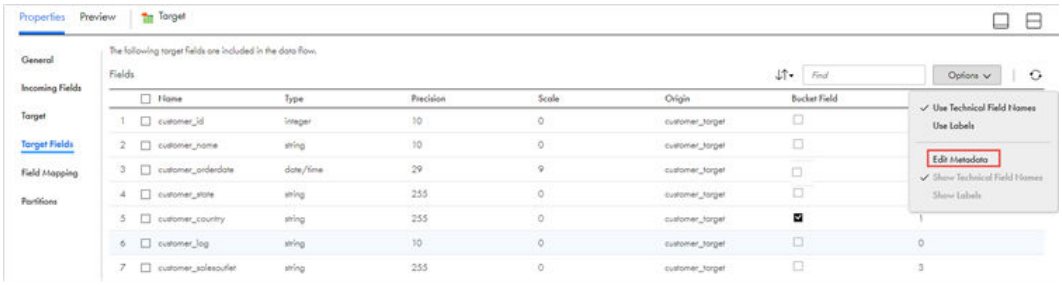
MongoDB V2 lookup properties

The following table describes the MongoDB V2 lookup object properties that you can configure in a Lookup transformation:

Property	Description
Connection	Name of the lookup connection. You can select an existing connection, create a new connection, or define parameter values for the lookup connection property. If you want to overwrite the lookup connection properties at run time, select the **Allow parameter to be overridden at run time** option.
Source Type	Type of the source object. Select Single Object or Parameter.
Parameter	A parameter file where you define values that you want to update without having to edit the task. Select an existing parameter for the lookup object or click **New Parameter** to define a new parameter for the lookup object. The **Parameter** property appears only if you select parameter as the lookup type. If you want to overwrite the parameter at run time, select the **Allow parameter to be overridden at run time** option. When the task runs, the Secure Agent uses the parameters from the file that you specify in the advanced session properties.
Lookup Object	Name of the lookup object.
Filter	Not applicable.
Sort	Not applicable.

Edit metadata in a MongoDB V2 source and target

You can edit field data types to customize the precision, scale and data types of fields. You can edit the field metadata in a Source or Target transformation to change simple data types, precision, and scale. To edit metadata, use the Edit Metadata option for the source and target fields on the **Fields** tab of the Properties panel.



Reading and writing hierarchical data in mappings

You can use the Hierarchy Processor transformation in a mapping to process data from complex data sources such as structs and arrays. You can read complex data types with hierarchical input and write as relational output. You can also read complex data types with relational input and write as hierarchical output.

When you configure a mapping to convert relational to hierarchical output, you can join data sources, group by and order by data fields, filter for specific information, and aggregate both the input and output data. When you configure a mapping to convert hierarchical to relational output, you can read from a single hierarchical data source and write to multiple relational data targets.

For more information about configuring a Hierarchy Processor transformation, see the Hierarchy Processor transformation section in *Transformations* in the Data Integration help.

Stop on errors

You can configure the maximum number of load job errors at the target that a mapping task can encounter before it stops. You can also capture the rejected rows in an error file.

The error threshold can include malformed rows and data conversion errors in the load job, but the errors resulting from update, delete, or merge queries are excluded.

You can set the error threshold value in the **Stop on errors** session property in a mapping task. You can set the value from 0 to 2147483647. Values less than 0, alphabets, and special characters are not allowed.

When you run a mapping task with bulk operations on the target, such as delete or load data, the mapping task executes operations in parallel instead of in sequence.

The following examples describe the mapping task behavior based on the error threshold you set:

- When you set the error threshold to the maximum value of 2147483647, the mapping task continues to run even after encountering errors.

- When you set the error threshold value to 0 or 1, the mapping task stops at the first error.
- When you set the error threshold value more than 1, the mapping task continues to run until the cumulative number of errors reaches or exceeds the configured threshold value, then it stops. Since the MongoDB V2 mapping task is executed in batch mode, the cumulative number of errors might exceed the configured threshold value before the mapping task stops.
  For example, consider the following scenario when you set the error threshold value to 4 in a mapping task that processes 5 batches:
  - Error count 1 after processing batch 1
  - Error count 2 after processing batch 2
  - Error count 5 after processing batch 3

  Here, the mapping task continues through batch 2 because the cumulative errors have not yet exceeded the threshold of 4. After batch 3 completes, the cumulative errors increase to 8, which exceeds the threshold, so the mapping task stops and doesn't proceed to the next batch 4 or batch 5.

You can't configure the stop on error functionality in mapping tasks that are based on mappings in advanced mode.

Capture rejected rows in an error file

When you write data to a MongoDB V2 target in a mapping task configured with an error threshold, you can capture the rejected rows in an error file.

Use the error file to understand why the Secure Agent didn't write data to the MongoDB V2 target.

The Secure Agent writes rejected rows to the error file for all target operations and for tasks that capture changed data.

1. Create a mapping task that uses the configured MongoDB V2 mapping.
2. On the **Runtime Options** tab, go to the **Advanced Session Properties** section.
3. From the **Session Property Name** list, add the following properties, and then set the values:
   a. Select the **Error Log Type** property and set the value to **Flat File**.
   b. Select the **Error Log File Directory** property and enter a directory for the error file.

      If you don't configure this property, the Secure Agent generates the error file in the following default directory:
      `<Secure Agent installation directory>/apps/Data_Integration_Server/data/error`

      To change the default directory, see the Change the default error file directory in the Secure Agent Knowledge article.

c. Select the **Error Log File Name** property, and enter a file name with the .bad extension for the error file.

Advanced Session Properties

Session Property Name	Session Property Value
Error Log Type	Flat File
Error Log File Directory	
Error Log File Name	insert_rejects.bad

If you don't configure this property, the Secure Agent names the error file in the following format:

```
<Mapping task ID>_error.csv
```

4. Click **Save** > **Run** to run the mapping task.

Rules and guidelines for generating error file

Consider the following rules and guidelines when you generate an error file for the rejected rows in a mapping task:

- The Secure Agent doesn't capture rows that the mapping task skips because the update columns contain null values.
- You can enter a directory in the **Error Log File Directory** property of a mapping task, in the Secure Agent, or in both settings. If you set a directory in both settings, the directory in the mapping task takes precedence.

Capture changed data from CDC sources in a mapping

You can use MongoDB V2 Connector to write the changed data from a CDC source, such as Microsoft SQL Server CDC, and write the changed data to a MongoDB V2 target in a mapping. You can't write the changed data to MongoDB from a CDC source in a mapping in advanced mode.

When you configure a mapping, add the CDC sources and then run the associated mapping task to write the changed data to MongoDB. If you define a column as required in the MongoDB V2 target table, map a column in the CDC source to the required column in the MongoDB V2 target in the mapping before you run the task.

When the mapping task processes the changed data from a CDC source, MongoDB V2 Connector creates a state table in MongoDB. When the changed data is received from the CDC source, MongoDB V2 Connector uploads the changed data to the staging table. Then, it generates a `Job_Id` and writes the `Job_Id` to the state table along with the restart information.

The connector then merges the stage table with the actual target table in MongoDB. Each time you run the mapping task, MongoDB V2 Connector creates the state table, if it does not exist, to store the state information.

If you stop a CDC mapping task enabled with recovery strategy, or if it terminates unexpectedly, you can also restart it from the beginning of the task or resume it from where the task last ended from the job details page.

For more information about restarting and resuming a mapping task, see *Tasks* in the Data Integration documentation.

Restrictions

Consider the following restrictions when you capture the changed data from CDC sources:

Source or lookup object in a CDC mapping

You can use a MongoDB V2 target in a CDC mapping to write data from CDC sources. You can't use a MongoDB V2 source or lookup in a CDC mapping.

Target created at run time

You can't write data from a CDC source to a MongoDB V2 target created at run time.

Configuring a mapping task to read from a CDC source

You can use MongoDB V2 Connector to capture the changed data from a CDC source and write the changed data to a MongoDB V2 target.

To capture the changed data from a CDC source, you need to add the CDC source in the mapping, and then run the associated mapping task to write the changed data to the MongoDB V2 target. You can also configure multiple pipelines in a single mapping to write the captured changed data to a MongoDB V2 target.

When you configure a mapping to write the changed data from a CDC source to MongoDB, you can configure the following advanced properties in the MongoDB V2 Target transformation:

- Update Mode
- Pre Query
- Post Query
- Additional Properties
- Forward Rejected Rows

Step 1. Configure the source

Configure a Source transformation to read from a CDC source, such as Microsoft SQL Server CDC.

1. In the Source transformation, specify a name and description in the general properties.
2. In the **Source** tab, select any configured CDC connection and specify the required source properties.

   It is recommended that the source object contains a primary key.

Step 2. Configure the target

Configure a Target transformation to write changed data from a CDC source to MongoDB.

You can only configure a single MongoDB V2 target transformation in a mapping to write changed data from a CDC source.

1. On the **Target** tab, perform the following steps to configure the target properties:
   a. In the **Connection** field, select the MongoDB V2 connection.
   b. In the **Object Type** field, select the type of the target object.

c. In the **Object** field, select the required target object.

d. In the **Operation** field, select **Data Driven**.

Note: Insert, update, upsert, and delete target operations are not applicable.

e. Configure the applicable advanced target properties for the CDC mode.

2. On the **Field Mapping** tab, map the incoming fields to the target fields. You can manually map an incoming field to a target field or automatically map fields based on the field names.

If you define a column as required in the MongoDB V2 target table, map a column in the CDC source to the required column in the MongoDB V2 target in the mapping.

Step 3. Configure the mapping task

After you create a mapping, add the mapping to a mapping task, and configure the advanced properties. Run the associated mapping task to write the changed data to MongoDB.

1. From the **Actions** menu, click **New Mapping Task**.

The **New Mapping Task** page appears.

2. In the **Definition** tab, enter the task name and select the configured mapping.
3. In the **CDC Runtime** tab, specify the required properties for the selected CDC source.

For more information about the **CDC Runtime** properties, see the source properties for the selected CDC source.

4. In the **Runtime Options** tab, add the following properties in the **Advanced Session Properties** section:

a. In the **Commit on End of File** field, select the value of the property as **No**.

b. In the **Commit Type** field, select the value of the property as **Source**.

c. In the **Recovery Strategy** field, select the value of the property as **Resume from last checkpoint**.

5. Click **Save** > **Run** the mapping task.

Alternatively, you can create a schedule that runs the mapping task on a recurring basis without manual intervention. You can define the schedule to minimize the time between mapping task runs. In **Monitor**, you can monitor the status of the logs after you run the task.

To improve performance, specify a higher commit interval for the Maximum Rows Per Commit property in the CDC Runtime page in the mapping task wizard. However, in case of failure, recovery takes more time for a higher commit interval.

Configure additional properties

You can use a MongoDB V2 connection to read from or write to Azure CosmosDB enabled with the MongoDB API.

When you run a mapping or a mapping in advanced mode, you can configure the following optional parameters in the advanced source or target properties to connect to Azure CosmosDB MongoDB API:

throughput or maxthroughput

Request units per second for a read or write operation. Expressed in RUs (Request Units). Throughput is a positive integer and a multiple of 100. You can modify the throughput for a read or write operation and depending on the operation, you can configure throughput or maxthroughput property under the advanced source or target properties. If you do not specify a value, the mapping utilizes the configured throughput for the collection.

For example, throughput=1000.

You can specify maxthroughput in place of throughput for a MongoDB collection. This property allows you to autoscale the throughput from 10% of maxthroughput to the RUs specified in maxthroughput. For example, if maxthroughput=2000 RUs, the throughput automatically scales from 200 to 2000 RUs based on usage.

You must either specify throughput or maxthroughput based on the throughput mode configured for the Azure Cosmos collection as Azure CosmosDB MongoDB API allows switching between these modes only from the Azure portal.

shardkey

A field or column that MongoDB uses to distribute documents among members of a shard cluster. Use the column as a shard key while creating a new collection. Configure shardkey property under the advanced target properties.

For example, shardkey=logid.

Rules and guidelines for MongoDB V2 mappings

Consider the following guidelines before you run a mapping:

- You cannot read or write MongoDB column or table names that have more than 225 characters.
- The source tables must not be empty. Otherwise, the mapping fails.
- If the source contains binary data, the data preview section shows jumbled text.
- When you run a mapping in advanced mode, the Secure Agent does not preserve the same order of columns in the MongoDB V2 target as in the source.
- When you perform the insert operation and specify a collection that does not exist, MongoDB automatically creates a new collection with that name.
- You cannot fetch the details of the rejected rows when you write data to a MongoDB V2 target.
- You cannot use complex data type columns as lookup conditions.
- Avoid mapping the *_id* column to optimize the write operation.
- When you run a mapping, you might view info logs in the session log under an error class reference. Here is an example where you can see the information message logged as an error under the log entry **READER_1_1_1> JAVA PLUGIN_1762**:

  ```
  READER_1_1_1> JAVA PLUGIN_1762 [2024-10-25 15:50:18.724] [ERROR] Oct 25, 2024 3:50:18 PM
  com.informatica.products.adapters.nosql.jdbc.Connection logBuildVersion INFO:
  com.informatica.products.adapters.nosql.jdbc.Connection: Build version: {OSNAME=***,
  HOSTNAME=***, RELEASE_VERSION_FLO=***, timestamp=***, componentversion=***}
  ```

  The error message here is an informational message about the build version of the connector and not an actual error. To view the encountered errors, refer to the error row file.

CHAPTER 4

Data type reference

Data Integration uses the following data types in mappings with MongoDB V2:

MongoDB V2 native data types

MongoDB V2 data types appear in the **Fields** tab for Source and Target transformations when you choose to edit metadata for the fields.

Transformation data types

Set of data types that appear in the transformations. They are internal data types based on ANSI SQL-92 generic data types, which the Secure Agent uses to move data across platforms. Transformation data types appear in all transformations in a mapping.

When Data Integration reads source data, it converts the native data types to the comparable transformation data types before transforming the data. When Data Integration writes to a target, it converts the transformation data types to the comparable native data types.

MongoDB V2 and transformation data types

The following table describes the MongoDB V2 data types that Data Integration supports and the corresponding transformation data types:

MongoDB V2 Data Types	Transformation Data Types	Range and Description
Array	String	An ordered list of zero or more elements of a single fixed type.
Array[1]	Array	-
Binary	Binary	1,67,77,216. Represented as a hexadecimal number.
Boolean	Integer	1=true, 0=false
BSON Null	String	1 to 104,857,600 characters
BSON Regular Expression	String	1 to 104,857,600 characters
BSONTimestamp	String	1 to 104,857,600 characters
Code (Javascript)	String	1 to 104,857,600 characters

MongoDB V2 Data Types	Transformation Data Types	Range and Description
Date	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to millisecond)
Decimal128	Decimal	Precision 1 to 38 digits. You cannot use decimal values with precision greater than 38.
Double	Double	1.7E +/- 308 (15 digits)
Int32	Integer	-2,147,483,648 to 2,147,483,647
Int64	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Integer	Integer	-2,147,483,648 to 2,147,483,647
Float	Double	1.7E +/- 308 (15 digits)
MaxKey	String	1 to 104,857,600 characters
MinKey	String	1 to 104,857,600 characters
Null[1]	Empty String	~~~~
Object	String	1 to 104,857,600 characters
Object[1]	Struct	-
ObjectID	String	1 to 104,857,600 characters
ObjectID[1]	Struct	-
String	String	1 to 104,857,600 characters
Symbol	String	1 to 104,857,600 characters
[1] Applies only to mappings in advanced mode.		

Rules and guidelines for data type reference

Consider the following rules and guidelines for MongoDB V2 data types:

- When you configure a mapping to create a MongoDB V2 target at runtime and run the mapping, Data Integration writes the data in transformation data type instead of the original data type.
- When you run a mapping, it succeeds even if the fields are mapped to different data types.
- When you map the primary key with any other key, the mapping fails.

INDEX

C

Cloud Application Integration community
  URL 5
Cloud Developer community
  URL 5

D

Data Integration community
  URL 5

I

Informatica Global Customer Support
  contact information 6
Informatica Intelligent Data Management Cloud
  web site 5

L

lookup
  database 23

M

maintenance outages 6
mapping
  MongoDB V2 sources 16
mapping in advanced mode
  MongoDB V2 targets 20
mapping tasks
  overview 16
mappings
  lookup overview 23
mappings *(continued)*
  lookup properties 23
MongoDB connector
  supported task and object types 7
MongoDB V2 connection
  rules and guidelines 29
MongoDB V2 connections
  overview 8
  properties 9
MongoDB V2 Connector
  overview 7
MongoDB V2 data types
  mapping to transformation data types 30
  overview 30
MongoDB V2 target transformation
  rules and guidelines 22

S

status
  IDMC 6
system status 6

T

trust site
  description 6

U

upgrade notifications 6

W

web site 5